Visual Basic Controls from Toolbox

VB 6.0 · OAVS Computer Teacher – Competitive Exam Revision Notes

Syllabus covered: 1. Properties 2. Methods 3. Events 4. Other Controls: Microsoft Windows Common Controls (6.0) 5. Microsoft Windows Common Controls-3 (6.0) 6. Control Arrays (adding controls at runtime & design time)

How to read: amber boxes = important exam points · red tables = commonly confused concepts · green boxes = short summaries. Code is VB 6.0.

The Big Picture: every Toolbox control is an object

A control dragged from the Toolbox is an **object**. Everything you can do with it falls under exactly three headings – **properties** (its data), **methods** (its actions) and **events** (what it reports back). The Toolbox holds **intrinsic (standard) controls** built into VB6 (Label, TextBox, CommandButton, ListBox, Timer, Frame, PictureBox …) and **ActiveX controls** stored in **.OCX** files that are added on demand (Section 4).

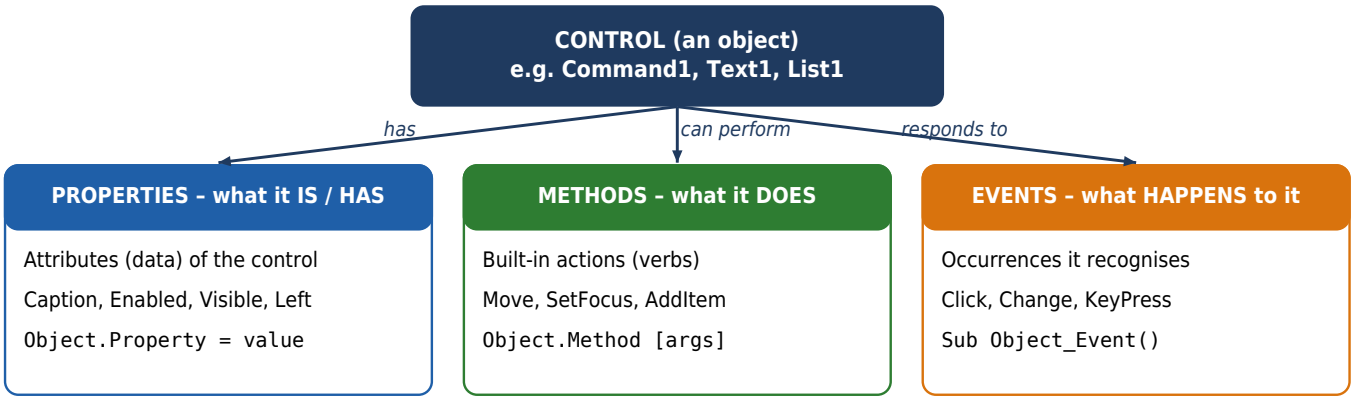


Fig. 1 – Control → Property / Method / Event relationship

1. Properties

1.1 Definition and purpose

A **property** is a named attribute of a control that stores one aspect of its **appearance, behaviour or data** (colour, size, text, state ...). Properties let you configure a control without writing its internal logic.

1.2 Setting and reading properties

A property can be set and read in **two ways**, depending on when it is used – at **design time** (while building the form) or at **run time** (while the program is running).

- **Design time** – select the control and edit its value in the Properties window (F4).
- **Run time** – assign in code using the dot notation: `Object.Property = value`. Read with `variable = Object.Property`.
- A property can itself be an object (e.g. `Font`), giving *sub-properties*: `Label1.Font.Bold = True`.

Program 1 – Setting and reading properties

```
Text1.Text = "Hello"            ' set
Command1.Enabled = False        ' set (disable the button)
Label1.Font.Bold = True         ' sub-property of Font
n = List1.ListCount             ' read
Text1.Left = Text1.Left + 100   ' read and set (twips)
```

▶ OUTPUT (expected result)

- **Text1** displays: `Hello`
- **Command1** becomes dimmed (disabled) and no longer responds to clicks.
- **Label1**'s text appears in **bold**.
- **n** receives the number of items in List1 (e.g. 3 if the list holds three items).
- **Text1** moves 100 twips (≈ 0.07 inch) to the right.

1.3 When can a property be used?

Category	Meaning	Examples
Read/write at both times	Set in Properties window and changed in code	Caption, Text, Enabled, Visible, BackColor, Left, Top
Design time only (read-only at run time)	Fixed once the program runs; assigning in code gives an error	Name, Index, MultiLine, ScrollBars, Sorted, MultiSelect, Style (ComboBox/ListBox)
Run time only (not in Properties window)	Exist only while the program runs	ListIndex, ListCount, Selected, NewIndex, SelStart, SelLength, SelText

1.4 Common properties

Property	Applies to	Purpose / values
`Name`	All controls	Identifier used in code; begins with a letter, up to 40 characters; **read-only at run time**. Default: Command1, Text1 ...
`Caption`	Label, CommandButton, CheckBox, OptionButton, Frame, Form	Text shown on the control. A **&** before a letter creates an access key (&Save → Alt+S).
`Text`	TextBox, ComboBox, ListBox	Text typed or selected (ListBox: read-only at run time).
`Enabled`	Most controls	True/False – can the control respond to the user? False = dimmed.
`Visible`	Most controls (not Timer)	True/False – shown or hidden.
`Left, Top, Width, Height`	Visible controls	Position and size in **twips** (1 inch = 1440 twips), measured from the container (Form, Frame or PictureBox).
`BackColor, ForeColor`	Most visible controls	Background / text colour: RGB(), QBColor() or constants such as vbRed.
`Font`	Controls with text	Font object – Name, Size, Bold, Italic, Underline, Strikethrough.
`TabIndex, TabStop`	Controls that take focus	TabIndex = Tab-key order (starts at 0); TabStop = False skips the control.
`ToolTipText`	Most visible controls	Hint displayed when the pointer rests on the control.
`Tag`	All controls	Free text for the programmer's own use; VB ignores it.
`Index`	Control-array elements only	Position of the element in its control array (Section 6).
`Appearance, BorderStyle`	Most visible controls	Appearance: 0 Flat, 1 3D. BorderStyle: 0 None, 1 Fixed Single. Read-only at run time for many controls (e.g. TextBox).

Property	Applies to	Purpose / values
`Parent, Container`	Visible controls	Parent = the form holding the control; Container = the Frame / PictureBox / Form it sits in.
`DragMode`	Visible controls	0 Manual (start with the Drag method), 1 Automatic (drag begins when the mouse is pressed).

1.5 Key properties of specific controls

Control	Important properties and values
CommandButton	`Default` = True (Enter triggers it; only one per form) · `Cancel` = True (Esc triggers it) · `Style` 0 Standard / 1 Graphical · Picture · Value
TextBox	`Text` · `MaxLength` (0 = no limit) · `MultiLine` · `ScrollBars` (0 None, 1 Horizontal, 2 Vertical, 3 Both; works only if MultiLine = True) · `PasswordChar` · `Locked` · SelStart / SelLength / SelText
Label	`Caption` · AutoSize · WordWrap · Alignment · BorderStyle · BackStyle (0 Transparent, 1 Opaque) – **no Text property, cannot receive focus**
CheckBox	`Value`: 0 = vbUnchecked, 1 = vbChecked, 2 = vbGrayed
OptionButton	`Value`: True / False – only one button per container (Form/Frame) can be True
ListBox / ComboBox	`List` · ListCount · `ListIndex` (–1 = nothing selected; first item = 0) · Selected() · Sorted · MultiSelect (ListBox: 0 None, 1 Simple, 2 Extended) · Style (ComboBox: 0 Dropdown Combo, 1 Simple Combo, 2 Dropdown List) · ItemData() · NewIndex · SelCount · Columns · TopIndex
Timer	`Interval` in milliseconds (0 = disabled; max 65,535) · `Enabled` – invisible at run time, no Visible property
HScrollBar / VScrollBar	`Min`, `Max`, `Value`, SmallChange (arrow click), LargeChange (click in the bar)
Frame	`Caption` – a **container**; controls drawn inside move with it; OptionButtons inside form their own group
PictureBox / Image	`Picture` · Image: Stretch · PictureBox: AutoSize, AutoRedraw – PictureBox is a container, Image is not
DriveListBox / DirListBox / FileListBox	Drive: `Drive` (selected drive letter) · Dir: `Path` (current folder) · File: `Path`, `Pattern` (e.g. "*.txt"), `FileName`, MultiSelect – the three are linked by code (see 3.4)
Shape / Line	Shape: `Shape` (0 Rectangle, 1 Square, 2 Oval, 3 Circle, 4 Rounded Rectangle, 5 Rounded Square), FillColor, FillStyle, BorderColor, BorderWidth · Line: X1, Y1, X2, Y2, BorderColor, BorderWidth – neither can take focus

★ IMPORTANT OAVS EXAM POINTS

- **Default properties** (usable without naming the property): TextBox → Text; Label → Caption; CheckBox, OptionButton, CommandButton, scroll bars → Value; ListBox, ComboBox → Text; PictureBox, Image → Picture; Timer → Enabled.
- **Name** and **Index** cannot be changed at run time; ListIndex = –1 means no selection; ListBox/ComboBox indexes are zero-based.
- Setting **CheckBox.Value**, **OptionButton.Value = True** or **CommandButton.Value = True** in code also fires the control's Click event.
- Default unit for Left/Top/Width/Height is the **twip**; 1 inch = 1440 twips (≈ 567 per cm).
- Pictures are assigned at run time with the **LoadPicture** function: `Picture1.Picture = LoadPicture("logo.bmp")`; `LoadPicture("")` clears the picture.
- A button's **Picture** is displayed only when **Style = 1 (Graphical)** – the same applies to CheckBox and OptionButton.

⚠ COMMONLY CONFUSED	Difference
Name vs Caption	Name = identifier used in code (fixed at run time). Caption = text the user sees (changeable).
Caption vs Text	Caption: Label, CommandButton, CheckBox, OptionButton, Frame, Form. Text: TextBox, ComboBox, ListBox. TextBox has no Caption; Label has no Text.
Enabled vs Visible	Enabled = False: still visible but dimmed and unresponsive. Visible = False: hidden completely.
Locked vs Enabled	TextBox Locked = True: looks normal, text can be selected/copied but not edited. Enabled = False: dimmed, cannot get focus.
Image vs PictureBox	Image: lightweight, has Stretch, not a container, no graphics methods. PictureBox: container, AutoSize, supports Print/Cls/Line/Circle.
Value (CheckBox vs OptionButton)	CheckBox Value is an integer 0/1/2; OptionButton Value is Boolean True/False.

✓ SHORT SUMMARY

- Properties are the **data/attributes** of a control, accessed as `Object.Property` at design time (Properties window) or run time (code).
- Some properties are design-time only (Name, Index, MultiLine …); some exist only at run time (ListIndex, SelText …).
- Know the key values: CheckBox Value 0/1/2, ListIndex –1, Timer Interval in ms, positions in twips.

2. Methods

2.1 Definition and purpose

A **method** is a built-in procedure belonging to a control that makes the control **perform an action** – move itself, take focus, add a list item, clear its contents. Where properties are *nouns/adjectives*, methods are *verbs*. Methods are available only in code (they never appear in the Properties window), and the set of methods depends on the control type.

2.2 Invoking methods

Program 2 – Invoking a method – the syntax forms

```
Object.Method [arg1, arg2, ...]          ' statement form - no brackets
result = Object.Method(arg1, arg2)       ' function form  - brackets, returns a value
Call Object.Method(arg1, arg2)           ' Call form      - brackets required
Command1.Move Left:=100, Top:=50         ' named arguments are allowed
```

▶ **OUTPUT (expected result)**

- Statement form, function form and Call form all run the method; the function form also returns a value into `result`.
- Last line: **Command1** jumps to Left = 100, Top = 50 twips; its width and height stay unchanged.

2.3 Important control methods

Method	Applies to	Syntax	Purpose
`Move`	All visible controls	`obj.Move Left [,Top] [,Width] [,Height]`	Changes position/size in one step (twips). Only **Left** is mandatory.
`SetFocus`	Controls that can take focus	`Text1.SetFocus`	Moves the focus to the control. Error if control is disabled or hidden.
`Refresh`	Most controls	`obj.Refresh`	Forces the control to repaint.
`ZOrder`	Visible controls	`obj.ZOrder [position]`	0 = bring to front (default), 1 = send to back.
`Drag`	Visible controls	`obj.Drag [action]`	Starts/ends a drag: 0 vbCancel, 1 vbBeginDrag, 2 vbEndDrag.
`AddItem`	ListBox, ComboBox	`lst.AddItem item [,index]`	Adds an entry; index is zero-based; omitted = at the end (or sorted position if Sorted = True).
`RemoveItem`	ListBox, ComboBox	`lst.RemoveItem index`	Deletes **one** entry by index.
`Clear`	ListBox, ComboBox	`lst.Clear`	Deletes **all** entries.
`Cls`	Form, PictureBox	`pic.Cls`	Erases text/graphics drawn at run time (not the bitmap set in Picture).
`Print`	Form, PictureBox	`pic.Print "text"`	Writes text on the object.
`PSet, Line, Circle`	Form, PictureBox	`pic.Circle (x, y), r`	Draw point, line, circle (graphics methods).
`TextWidth, TextHeight`	Form, PictureBox	`w = pic.TextWidth("Hi")`	Return the size a string would occupy.

2.4 Examples

Program 3 – List methods – AddItem, RemoveItem, Clear, SetFocus, Move, Print

```
List1.AddItem "Bhubaneswar"        ' added at the end
List1.AddItem "Cuttack", 0         ' inserted at the top (index 0)
List1.RemoveItem 1                 ' removes the 2nd item (index 1)
Text1.SetFocus                     ' cursor goes to Text1
Command1.Move 500, 300             ' Left = 500, Top = 300 twips
Picture1.Print "Total = " & n      ' text drawn on the PictureBox
List1.Clear                        ' empties the list
```

▶ **OUTPUT (expected result)**

- Assuming List1 was empty at the start:

```
After line 1   List1:  Bhubaneswar
After line 2   List1:  Cuttack, Bhubaneswar       (index 0 = top)
After line 3   List1:  Cuttack                    (index 1 removed)
After line 4   the cursor blinks inside Text1
After line 5   Command1 is now at Left = 500, Top = 300 twips
After line 6   Picture1 shows:  Total = 25        (if n = 25)
After line 7   List1 is empty
```

Note: the Timer control has **no methods**; Label, Shape and Line have only the basic ones (Move, Refresh, ZOrder, Drag). Common-control collections (Nodes, ListItems, Panels ...) have their own Add / Remove / Clear methods – see Section 4.

★ IMPORTANT OAVS EXAM POINTS

- **AddItem** index is zero-based; **RemoveItem** removes one item, **Clear** removes all.
- **SetFocus** fails (run-time error) if the control is disabled or not visible.
- **Move** syntax: only the first argument (Left) is compulsory; **ZOrder 0** = front, **ZOrder 1** = back.
- Graphics methods (Print, Cls, Line, Circle, PSet) work on **Form and PictureBox**, not on TextBox or Label.

⚠ COMMONLY CONFUSED	Difference
Property vs Method vs Event	Property = attribute (*Text1.Text* = "*A*"). Method = action you *call* (*Text1.SetFocus*). Event = occurrence the control *reports* (*Text1_Change*). You set properties, call methods, and respond to events.
SetFocus vs GotFocus	SetFocus is a **method** (you move the focus). GotFocus is an **event** (fires after the control receives focus).
Clear vs Cls	Clear empties a ListBox/ComboBox. Cls erases drawn output of a Form/PictureBox.
RemoveItem vs Clear	RemoveItem needs an index and removes one entry; Clear takes no argument and removes everything.
Move vs Left/Top	Move changes position/size in one call; Left, Top, Width, Height are properties changed one at a time.

✓ SHORT SUMMARY

- Methods are **actions** built into a control, invoked as `Object.Method args` in code only.
- Remember the core set: Move, SetFocus, Refresh, ZOrder, Drag; AddItem / RemoveItem / Clear for lists; Cls / Print / Line / Circle for Form and PictureBox.

3. Events

3.1 Definition, event-driven programming and event procedures

An **event** is an action recognised by a control. It can be caused by the **user** (click, key press, mouse move), the **system** (Timer interval elapsed) or **code** (changing Text raises Change). An **event procedure** is the block of code that VB runs automatically when its event occurs.

Event-driven programming: instead of running top-to-bottom in a fixed order (procedural style), a VB application sits idle and executes only the event procedures that the user or system triggers, **in whatever order they occur**. Controls with no matching event procedure simply ignore the event.

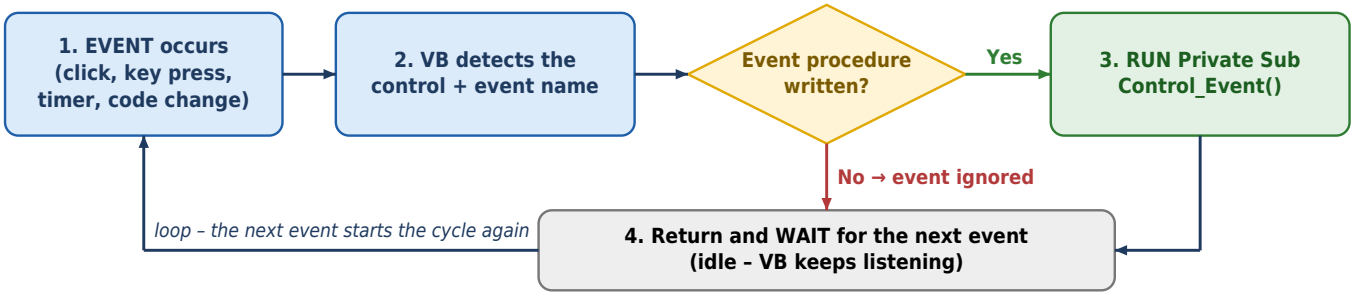


Fig. 2 – Event-driven programming flow

3.2 Event procedure syntax

Program 4 – Event procedure – syntax and example

```
Private Sub ControlName_EventName([arguments])
    ' statements executed when the event occurs
End Sub

Private Sub cmdGreet_Click()
    lblMsg.Caption = "Hello, " & txtName.Text
End Sub
```

▶ OUTPUT (expected result)

- The first block is only the general form. For the example, if **txtName** contains `Anil` and the user clicks **cmdGreet**:

```
txtName :  Anil
lblMsg  :  Hello, Anil          <- appears when cmdGreet is clicked
```

- Procedure name = **control Name + underscore + event name**. Double-clicking a control opens its **default event**; any other event is chosen from the Object / Procedure boxes of the Code window.
- If a control is **renamed after** code was written, the old procedure is no longer linked to it and will not run.
- An event procedure can be called like any procedure (`Call cmdGreet_Click`) – this runs the code but does not generate a real click.

Control	Default event	Control	Default event
CommandButton	Click	ListBox	Click
TextBox	Change	ComboBox	Change
Label, CheckBox, OptionButton	Click	HScrollBar / VScrollBar	Change
PictureBox, Image, Frame	Click	Timer	Timer

3.3 Common events

Event	Occurs when …	Typical controls	Arguments
`Click`	Control is clicked (also when an item is selected or Value changes)	Most controls	–
`DblClick`	Control is double-clicked	Most (not CommandButton)	–
`Change`	Contents/value change	TextBox, Label, ComboBox, scroll bars, PictureBox	–
`KeyPress`	A key producing an ASCII character is pressed	Controls with focus	`KeyAscii`
`KeyDown / KeyUp`	Any key (incl. F-keys, arrows) goes down / up	Controls with focus	`KeyCode, Shift`
`MouseDown / MouseUp`	Mouse button pressed / released	Most controls	`Button, Shift, X, Y`
`MouseMove`	Pointer moves over the control	Most controls	`Button, Shift, X, Y`
`GotFocus / LostFocus`	Control receives / loses focus	Controls that can take focus (not Label, Image, Shape, Line, Timer)	–
`Timer`	Interval milliseconds have elapsed	Timer	–
`Scroll`	Thumb is dragged (continuously)	HScrollBar, VScrollBar	–
`DragDrop / DragOver`	Dragged object is dropped on / moved over	Most visible controls	`Source, X, Y`
`DropDown`	Drop-down list opens	ComboBox	–
`Resize`	Size of the control changes	PictureBox	–
`Change, PathChange, PatternChange`	Selected drive / folder / file pattern changes	DriveListBox, DirListBox (Change); FileListBox (PathChange, PatternChange)	–

Argument values: `Button`: 1 = left (vbLeftButton), 2 = right (vbRightButton), 4 = middle. `Shift`: 1 = Shift, 2 = Ctrl, 4 = Alt. `KeyAscii`: Enter = 13, Esc = 27; setting `KeyAscii = 0` **cancels** the key (arguments are passed by reference, so changing KeyAscii changes the key actually received). Button and Shift are bit-masks: test with `If Shift And vbCtrlMask Then` (vbShiftMask 1, vbCtrlMask 2, vbAltMask 4).

3.4 Event sequences and examples

Action	Order of events
Click a CommandButton	`MouseDown → MouseUp → Click`
Type a character in a TextBox	`KeyDown → KeyPress → Change → KeyUp`
Tab into / out of a TextBox	`GotFocus` / `LostFocus`

Program 5 – Key, mouse, Timer and Drive/Dir/File events

```
Private Sub txtName_KeyPress(KeyAscii As Integer)
    If KeyAscii = 13 Then cmdOK.SetFocus        ' Enter moves to the button
End Sub

Private Sub txtName_KeyDown(KeyCode As Integer, Shift As Integer)
    If KeyCode = vbKeyF1 Then lblHelp.Caption = "F1 pressed"   ' KeyDown sees F-keys
End Sub

Private Sub picBox_MouseDown(Button As Integer, Shift As Integer, X As Single, Y As Single)
    If Button = vbRightButton Then lblInfo.Caption = "Right click at " & X & "," & Y
End Sub

Private Sub tmrClock_Timer()           ' fires every Interval ms (e.g. 1000)
    lblTime.Caption = Time
End Sub

Private Sub Drive1_Change()            ' linking Drive -> Dir -> File list boxes
    Dir1.Path = Drive1.Drive
End Sub
Private Sub Dir1_Change()
    File1.Path = Dir1.Path
End Sub
```

▶ OUTPUT (expected result)

```
What the user does                        Result on the form
----------------------------------------  ------------------------------------------
Presses Enter in txtName (KeyAscii = 13)  Focus moves to cmdOK
Presses F1 in txtName                     lblHelp shows:  F1 pressed
Right-clicks picBox at X = 1215, Y = 840  lblInfo shows:  Right click at 1215,840
Every 1000 ms (tmrClock, Interval = 1000) lblTime shows the time, e.g.  10:42:07 AM
Selects drive D: in Drive1                Dir1 lists the folders of D:Selects a folder in Dir1                File1
```

★ IMPORTANT OAVS EXAM POINTS

- **KeyPress** receives an ASCII code (*KeyAscii*) and ignores F-keys and arrows; **KeyDown/KeyUp** receive a key code and detect every key.
- **ListBox** has no Change event – use **Click**. **TextBox** fires **Change** on every edit, including edits made by code.
- Placing `Text1.Text = ""` inside `Text1_Change` triggers Change again → endless recursion (“Out of stack space”).
- **Timer** works only when **Enabled = True** and **Interval > 0**; Interval is in **milliseconds** (1000 = 1 second).
- Event procedures are **Private** by default and always begin with the control’s Name; extra first argument **Index** appears for control arrays.

⚠ COMMONLY CONFUSED	Difference
Click vs MouseDown	MouseDown fires first the moment the button goes down and tells *which* button and position; Click fires after MouseUp and carries no arguments.
KeyPress vs KeyDown	KeyPress: character keys only, gives ASCII. KeyDown: all keys, gives KeyCode and Shift state.
Change vs Click	Change = content changed (TextBox, ComboBox text, scroll bar). Click = control clicked / item chosen (ListBox, CheckBox, OptionButton).
Scroll vs Change	Scroll fires continuously while the thumb is dragged; Change fires after the value has changed.
Event vs Event procedure	Event = the occurrence (Click). Event procedure = your code (`cmdOK_Click`) that handles it.
GotFocus vs SetFocus	GotFocus is an event; SetFocus is a method that causes it.

✓ SHORT SUMMARY

- An **event** is something that happens to a control; the matching **event procedure** `Private Sub Control_Event()` runs automatically.
- VB programs are **event-driven**: the user decides the order of execution. Learn the common events and their arguments (KeyAscii, KeyCode, Button, X, Y).

4. Other Controls

4.1 What they are and how to add them

Beyond the standard Toolbox controls, VB 6.0 offers **ActiveX controls** stored in **.OCX** files. They are added from **Project → Components (Ctrl+T)** by ticking the library; the controls then appear at the bottom of the Toolbox. The Windows **Common Controls** give programs the standard Windows look.

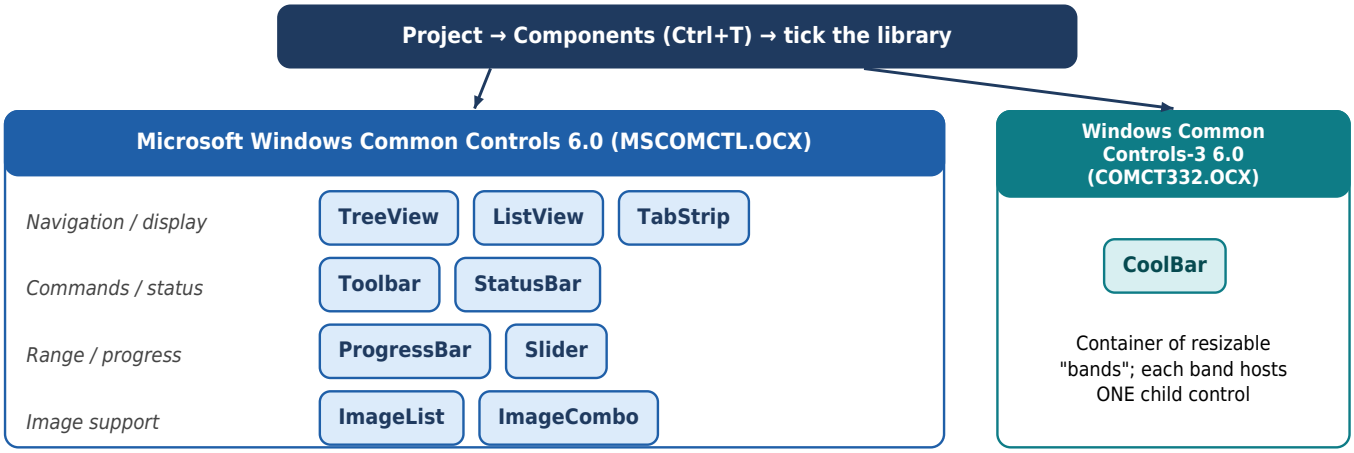


Fig. 3 – Organisation of the two Common Controls libraries in this syllabus

4.2 Microsoft Windows Common Controls (6.0) – MSCOMCTL.OCX

Control	Purpose	Key members	Key events
ImageList	Invisible store of pictures used by other controls	`ListImages` collection (index starts at 1), ImageWidth/ImageHeight, MaskColor	–
Toolbar	Row of buttons giving quick access to commands	`Buttons` collection, ImageList, ShowTips, Wrappable; Button: Key, Caption, Style, Image	`ButtonClick(Button)`, ButtonMenuClick
StatusBar	Information strip at the bottom of a window (text, caps/num lock, date, time)	`Panels` collection (max 16), Style: 0 sbrNormal / 1 sbrSimple, SimpleText; Panel.Style, Panel.Text	PanelClick
ProgressBar	Shows how much of a task is complete (output only)	`Min` (0), `Max` (100), `Value`, Orientation, Scrolling	–
Slider	User drags a thumb to pick a value (input)	Min, Max, Value, SmallChange, LargeChange, TickStyle, TickFrequency, SelectRange	Change, Scroll
TabStrip	Tabs for switching between sets of controls	`Tabs` collection, SelectedItem, Style, MultiRow, Placement	Click, BeforeClick
TreeView	Hierarchical list of nodes (like Windows Explorer folders)	`Nodes` collection; Node: Text, Key, Parent, Children, Expanded, Image; Style, LineStyle, Checkboxes, LabelEdit	NodeClick(Node), Expand, Collapse
ListView	List of items shown as icons, small icons, list or report (columns)	`ListItems`, `ColumnHeaders`, `View`: 0 lvwIcon, 1 lvwSmallIcon, 2 lvwList, 3 lvwReport; GridLines, FullRowSelect, Icons / SmallIcons	ItemClick(Item), ColumnClick
ImageCombo	ComboBox whose items can have pictures	`ComboItems` collection, ImageList, Text, SelectedItem	Click, Change

Working with them

- Items of these controls live in **collections** (Nodes, ListItems, Panels, Buttons, Tabs, ListImages). Collections are **1-based**, support **Add / Remove / Clear**, and each member may have a unique string **Key**.
- To show pictures, first fill an **ImageList**, then link it: `Set TreeView1.ImageList = ImageList1` (add all images *before* linking – a bound ImageList cannot be modified).
- **TreeView** relationships for `Nodes.Add(relative, relationship, key, text, image)`: tvwFirst 0, tvwLast 1, tvwNext 2, tvwPrevious 3, **tvwChild 4**. A Key must be unique and cannot be a number.
- **TabStrip** is **not** a container: put the controls of each tab in a Frame (usually a control array) and show the one matching the selected tab.
- At **design time** these controls are set up through their **Property Pages** (right-click → Properties, or "(Custom)" in the Properties window): Buttons, Panels, Tabs, Images, Bands tabs.

- Other useful members: Toolbar / StatusBar **Align** (top / bottom of the form); TreeView **Nodes.Remove, Nodes.Clear, SelectedItem**, Node.**FullPath**; ListView **SelectedItem**, **Sorted** / **SortKey**, ListItems.**Remove**.
- **ImageList:** ImageWidth / ImageHeight must be chosen **before** the first picture is added (read-only afterwards). **StatusBar panel AutoSize:** 0 sbrNoAutoSize, 1 sbrSpring (takes free space), 2 sbrContents (fits its text).

Add-method syntax and main click event of each collection

Control	Add syntax (arguments optional)	Main event signature
Toolbar	`Buttons.Add([index],[key],[caption],[style],[image])`	`ButtonClick(ByVal Button As Button)`
StatusBar	`Panels.Add([index],[key],[text],[style],[picture])`	`PanelClick(ByVal Panel As Panel)`
TabStrip	`Tabs.Add([index],[key],[caption],[image])`	`Click · BeforeClick(Cancel As Integer)`
TreeView	`Nodes.Add([relative],[relationship],[key],[text],[image],[selectedimage])`	`NodeClick(ByVal Node As Node)`
ListView	`ListItems.Add([index],[key],[text],[icon],[smallicon])`	`ItemClick(ByVal Item As ListItem)`
ListView	`ColumnHeaders.Add([index],[key],[text],[width],[alignment],[icon])`	`ColumnClick(ByVal ColumnHeader As ColumnHeader)`
ImageList	`ListImages.Add([index],[key],[picture])`	–
ImageCombo	`ComboItems.Add([index],[key],[text],[image],[selimage],[indentation])`	`Click · Change`

Program 6 – ProgressBar – showing progress

```
' ProgressBar
ProgressBar1.Min = 0: ProgressBar1.Max = 100
For i = 1 To 100: ProgressBar1.Value = i: Next i      ' Value must lie between Min and Max
```

▶ OUTPUT (expected result)

```
Start :  [                    ]    0%
Middle:  [##########          ]   50%
End   :  [####################]  100%
```

- The loop runs almost instantly, so the bar seems to jump to full; put the Value change in a Timer event to watch it fill gradually. A Value outside Min–Max causes a run-time error.

Program 7 – StatusBar – panels with text and date

```
' StatusBar
StatusBar1.Panels(1).Text = "Ready"
StatusBar1.Panels(2).Style = sbrDate                  ' panel shows the system date
```

▶ OUTPUT (expected result)

```
+-----------------------------------+--------------+
| Ready                             | 24-10-2026   |
+-----------------------------------+--------------+
```

- Panel 2 shows the current system date (format follows the computer's regional settings).

Program 8 – TreeView – adding root and child nodes

```
' TreeView - Odisha > Bhubaneswar
Dim nd As Node
Set nd = TreeView1.Nodes.Add(, , "od", "Odisha")                        ' root node
Set nd = TreeView1.Nodes.Add("od", tvwChild, "bbsr", "Bhubaneswar")     ' child of "od"
```

▶ OUTPUT (expected result)

```
[+] Odisha                <- collapsed at first (unless Expanded = True is set)
[-] Odisha                <- after clicking the + sign
     |-- Bhubaneswar
```

Program 9 – ListView – report view with columns

```
' ListView in report view
ListView1.View = lvwReport
ListView1.ColumnHeaders.Add , , "Name", 1500
ListView1.ColumnHeaders.Add , , "Class", 1000
Dim li As ListItem
Set li = ListView1.ListItems.Add(, , "Anil")          ' first column
li.SubItems(1) = "X"                                  ' second column
```

▶ OUTPUT (expected result)

```
Name          Class
------------  ----------
Anil          X
```

Program 10 – Toolbar – handling a button click

```
' Toolbar button click (identify the button by its Key)
Private Sub Toolbar1_ButtonClick(ByVal Button As MSComctlLib.Button)
    If Button.Key = "save" Then MsgBox "Saved"
End Sub
```

▶ OUTPUT (expected result)

- Clicking the Toolbar button whose Key is `save` displays a message box with the text `Saved`; other buttons do nothing.

Program 11 – TabStrip with a Frame control array

```
' TabStrip + Frame control array: show only the frame of the selected tab
Private Sub TabStrip1_Click()
    Dim i As Integer
    For i = fraTab.LBound To fraTab.UBound
        fraTab(i).Visible = (i = TabStrip1.SelectedItem.Index - 1)
    Next i
End Sub
```

▶ OUTPUT (expected result)

```
Click Tab 1  ->  only fraTab(0) is visible
Click Tab 2  ->  only fraTab(1) is visible
Click Tab 3  ->  only fraTab(2) is visible
```

★ IMPORTANT OAVS EXAM POINTS

- **Common Controls 6.0 = MSCOMCTL.OCX** (TabStrip, Toolbar, StatusBar, ProgressBar, TreeView, ListView, ImageList, Slider, ImageCombo – 9 controls).
- Version 5.0 of the common controls is COMCTL32.OCX; version **6.0** is MSCOMCTL.OCX.
- **ImageList** is invisible at run time and never used alone; its ListImages index starts at **1**.
- StatusBar holds up to **16 panels**; panel style constants: sbrText 0, sbrCaps 1, sbrNum 2, sbrIns 3, sbrScrl 4, sbrTime 5, sbrDate 6, sbrKana 7.
- ListView shows a table only when **View = lvwReport (3)** and ColumnHeaders are added; sub-items are set through **SubItems(n)**.
- Items in these collections are **1-based**, unlike ListBox/ComboBox items (0-based).

⚠ COMMONLY CONFUSED	Difference
ProgressBar vs Slider	ProgressBar only *displays* progress (set by code). Slider lets the *user* choose a value (Change/Scroll events).
Slider vs ScrollBar	Slider has tick marks and a selection range and is used to pick a value; scroll bars scroll content. Both use Min/Max/Value.
ImageList vs ImageCombo	ImageList stores pictures (invisible). ImageCombo is a visible drop-down that *uses* an ImageList to show pictures with its items.
TreeView vs ListView	TreeView = hierarchy of Nodes (expand/collapse). ListView = flat list of ListItems in icon, list or report view.
TabStrip vs Frame	TabStrip only supplies the tabs and is not a container; Frame holds controls. They are used together.

✓ SHORT SUMMARY

- Common Controls 6.0 (MSCOMCTL.OCX) = nine controls that give programs the standard Windows look.
- Most of them manage items through 1-based collections with Add/Remove/Clear and use an ImageList for pictures.

5. Microsoft Windows Common Controls-3 (6.0) – COMCT332.OCX

5.1 Overview and purpose

This library supplies a single control – the **CoolBar**. It is added like the other ActiveX controls: **Project → Components (Ctrl+T)** → tick *Microsoft Windows Common Controls-3 (6.0)*. A CoolBar is a **container** that holds user-configurable toolbars like those in Internet Explorer. It is made of one or more resizable regions called **bands**; each band can host **one child control** (a Toolbar, ComboBox, PictureBox …) and has a **gripper** so the user can drag and resize it.

5.2 CoolBar – main members

Item	Details
Main collection	Bands – each Band has Caption, Image, Child, NewRow, MinWidth, MinHeight, Key, Visible
CoolBar properties	Orientation (horizontal/vertical), BandBorders, FixedOrder, ImageList, Picture, Align, VariableHeight
Typical use	Draw the child control **inside** the CoolBar, then assign it as a band's Child (Property Pages → Bands, or in code)
Requirement	Needs Internet Explorer 3.0 or later on the development and target computers

5.3 Example

Program 12 – CoolBar – adding a band with a Toolbar

```
Dim bnd As Band
Set bnd = CoolBar1.Bands.Add()          ' add a band
bnd.Caption = "Tools"
Set bnd.Child = Toolbar1                ' Toolbar1 was drawn inside CoolBar1
```

▶ OUTPUT (expected result)

- CoolBar1 now shows a band captioned **Tools** with a gripper at its left edge; Toolbar1 appears inside that band:

```
+-------------------------------------------------+
| :: Tools   [ Toolbar1 buttons appear here ]     |
+-------------------------------------------------+
```

- The user can drag the gripper (::) to move or resize the band.

★ IMPORTANT OAVS EXAM POINTS

- **Common Controls-3 = COMCT332.OCX**; the only control it supplies is the **CoolBar**.
- A CoolBar is a **container of bands** (Bands collection); each band holds **one child control** (Band.Child).
- The child control must be drawn **inside** the CoolBar before it is assigned to a band.
- Every band has a **gripper** so the user can move and resize it; **FixedOrder** controls whether bands may be re-ordered.
- CoolBar needs **Internet Explorer 3.0 or later**.

⚠ COMMONLY CONFUSED	Difference
Toolbar vs CoolBar	Toolbar = one row of buttons (Buttons collection). CoolBar = container of movable bands that can host a Toolbar (Bands collection).
Band vs child control	A Band is the movable strip (Caption, gripper); its Child is the control placed on it (Toolbar, ComboBox …). One child per band.
Common Controls 6.0 vs Common Controls-3	Common Controls 6.0 (MSCOMCTL.OCX) has nine controls; Common Controls-3 (COMCT332.OCX) has only CoolBar.

✓ SHORT SUMMARY

- Common Controls-3 (6.0) = **COMCT332.OCX** = the **CoolBar** control only.
- CoolBar = container of resizable **bands**, each hosting one child control such as a Toolbar.

6. Control Arrays (adding controls at runtime & design time)

6.1 Definition, purpose and indexing

A **control array** is a group of controls of the **same type** that share the **same Name** and the **same event procedures**. Each member (*element*) is told apart by its **Index** property, written in brackets: `cmdNum(0)`, `cmdNum(1)` …

- **Purpose:** share one block of code among many controls, create controls **at run time**, loop over related controls, and save names (a form allows only 254 control names; a whole array counts as one).
- **Indexing:** Index is a non-negative integer, the first element is usually **0**, the maximum is **32767**; indexes need not be consecutive. Index is **read-only at run time**.
- **Array properties:** `LBound` (lowest index), `UBound` (highest index), `Count` (number of elements – differs from UBound + 1 when there are gaps).
- **Shared events:** every event procedure gains an extra first argument, `Index As Integer`, telling which element raised the event.

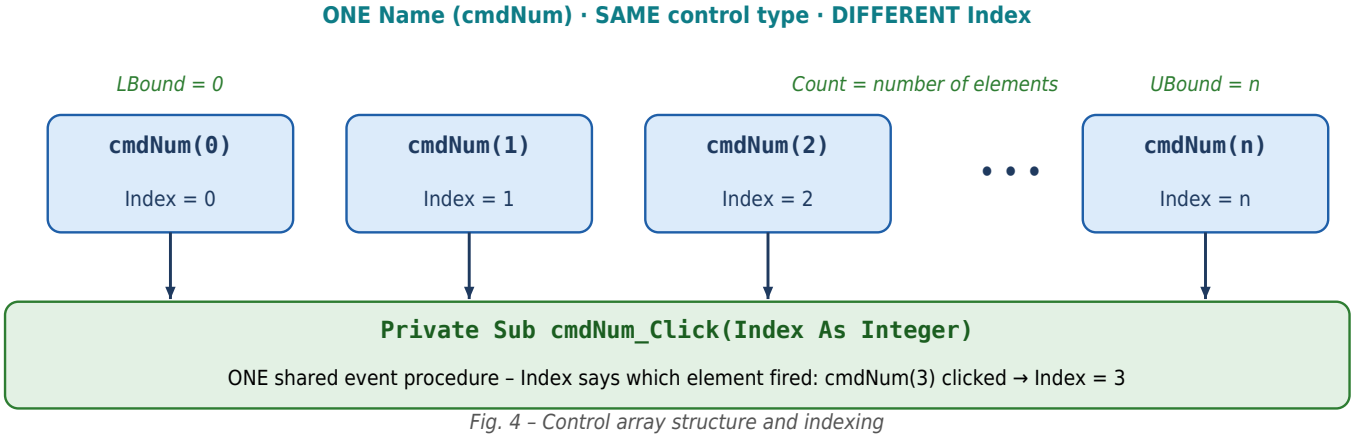


Fig. 4 – Control array structure and indexing

6.2 Creating a control array at design time

- **Copy and paste:** select a control, press Ctrl+C then Ctrl+V on the form and answer **Yes** to "You already have a control named …. Do you want to create a control array?".
- **Same Name:** give a second control of the same type the same Name and confirm the prompt.
- **Index property:** set the Index of a single control to 0 – it becomes a one-element array. Clearing the Index of a control turns it back into an ordinary control.

Program 13 – Control array – one shared event procedure

```
Private Sub cmdNum_Click(Index As Integer)      ' one procedure for all digit buttons 0-9
    txtDisplay.Text = txtDisplay.Text & Index   ' Index = the digit on the clicked button
End Sub

Private Sub optColor_Click(Index As Integer)    ' option-button array
    lblChoice.Caption = "Option " & Index & " selected"
End Sub
```

▶ OUTPUT (expected result)

```
Click digit buttons  3, 7, 2          ->  txtDisplay shows:  372        (each Index is appended)
Click the third option button         ->  lblChoice shows:   Option 2 selected   (Index = 2)
```

6.3 Adding controls at run time

At least **one element must already exist** (made at design time, e.g. Index = 0). New elements are created with the **Load** statement and removed with **Unload**. A loaded element copies the properties of the first element but is **invisible** and sits on top of it, so set its position and make it visible. New elements automatically use the array's existing event procedures.

Program 14 – Control array – adding and removing elements at run time

```
Private Sub cmdAdd_Click()                    ' txtBox(0) exists from design time
    Dim n As Integer
    n = txtBox.UBound + 1                     ' next free index
    Load txtBox(n)                            ' create the new element
    txtBox(n).Top = txtBox(n - 1).Top + 400   ' place below the previous one
    txtBox(n).Text = "Box " & n
    txtBox(n).Visible = True                  ' loaded controls start hidden
End Sub

Private Sub cmdRemove_Click()
    If txtBox.UBound > 0 Then Unload txtBox(txtBox.UBound)   ' only run-time elements
End Sub

Private Sub cmdClear_Click()                  ' visiting every element
    Dim i As Integer
    For i = txtBox.LBound To txtBox.UBound
        txtBox(i).Text = ""
    Next i
End Sub
```

▶ OUTPUT (expected result)

```
Start (design time)        txtBox(0)
After 2 clicks on cmdAdd   txtBox(0)
                           txtBox(1)  "Box 1"   (400 twips below txtBox(0))
                           txtBox(2)  "Box 2"   (400 twips below txtBox(1))
After cmdRemove            txtBox(2) disappears; txtBox(0) and txtBox(1) remain
After cmdClear             every remaining text box becomes empty
```

- If only txtBox(0) exists, **cmdRemove** does nothing (UBound = 0), so the design-time element is never unloaded.

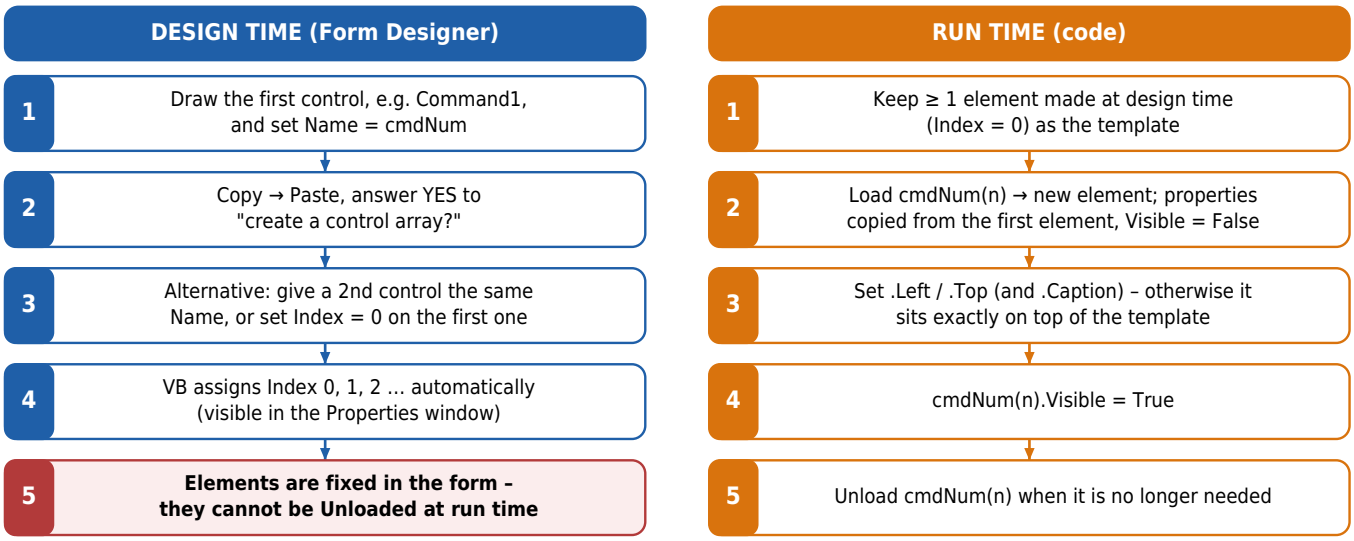


Fig. 5 – Design-time vs run-time creation of control-array elements

Feature	Design time	Run time
How	Copy–paste, same Name, or Index property in the Form Designer	Load ctrl(n) / Unload ctrl(n) in code
Index assigned by	VB automatically (0, 1, 2 ...)	You – any unused non-negative number
Appearance	Visible as drawn	Starts hidden – set Left/Top and Visible = True
Number of elements	Fixed while the program runs	Can grow and shrink while the program runs
Removal	Cannot be Unloaded at run time	Unload removes the element

6.4 Adding a control at run time without a control array – Controls.Add (VB 6.0)

VB 6.0 can also create a control purely in code, even when no element exists at design time, through the form's **Controls** collection. The variable is declared **WithEvents** so that the new control's events can be handled; the control starts **invisible**.

Program 15 – Controls.Add – creating a control without a control array

```
Private WithEvents cmdNew As CommandButton          ' General Declarations of the form

Private Sub Form_Load()
    Set cmdNew = Controls.Add("VB.CommandButton", "cmdNew")    ' ProgID, Name
    cmdNew.Caption = "Click me"
    cmdNew.Move 600, 600, 1500, 400
    cmdNew.Visible = True
End Sub

Private Sub cmdNew_Click()                          ' handled like any event procedure
    MsgBox "New button clicked"
End Sub
```

▶ OUTPUT (expected result)

- When the form loads, a button captioned **Click me** appears at Left = 600, Top = 600 twips (size 1500 × 400).
- Clicking it shows a message box: `New button clicked`

- ProgIDs of intrinsic controls: `"VB.TextBox"`, `"VB.Label"`, `"VB.CommandButton"` .... Remove with `Controls.Remove "cmdNew"` (not Unload).

★ IMPORTANT OAVS EXAM POINTS

- A control array needs the **same type**, the **same Name** and a unique **Index**; the Index property is the only thing that distinguishes the elements.
- **Load** creates a new element (error if that index is already loaded); **Unload** removes only elements that were created at run time.
- Maximum index is **32767**; a form can hold at most **254** control names – control arrays help to stay below this limit.
- New run-time elements are **invisible** and overlap the first element until you set **Left/Top** and **Visible = True**.
- Event procedures of a control array receive **Index As Integer** as their first argument; `Select Case Index` or `Index` directly tells which element fired.

⚠ COMMONLY CONFUSED	Difference
Control array vs array variable	A control array holds *controls* (objects on the form) and is created by Name/Index or Load; an array variable (Dim a(5)) holds data values.
Index vs ListIndex vs TabIndex	Index = position in a control array. ListIndex = selected item of a ListBox/ComboBox. TabIndex = Tab-key order.
Load vs Visible	Load creates the element in memory; Visible = True actually shows it. Unload destroys it; Visible = False only hides it.
Count vs UBound	Count = number of elements. UBound = highest Index. They differ by one when indexes start at 0, and further if there are gaps.
Control array vs Controls.Add	Control array: elements share Name and events, need an Index, use Load / Unload. Controls.Add: one new control with its own Name, declared WithEvents, removed with Controls.Remove.

✓ SHORT SUMMARY

- A control array = same-type controls + same Name + different **Index** + shared event procedures with an **Index** argument.
- Create at **design time** by copy/paste, same Name or Index; add at **run time** with **Load** (then position and make Visible) and remove with **Unload**.

Quick Revision

Key definitions

Term	One-line definition
Property	Attribute of a control (appearance, behaviour, data); `Object.Property = value`
Method	Built-in action a control can perform; `Object.Method [args]`
Event	Action recognised by a control (user, system or code)
Event procedure	`Private Sub Control_Event()` – code run automatically when the event occurs
Event-driven programming	Execution order is decided by events, not by a fixed sequence
ActiveX (OCX) control	Reusable control in an .OCX file, added via Project → Components
Control array	Same-type controls sharing one Name and event procedures, identified by Index

Important properties, methods and events

Properties	Methods	Events
Name, Caption, Text, Value, Enabled, Visible, Left/Top/Width/Height, BackColor/ForeColor, Font, TabIndex, TabStop, Tag, Index, Interval, MaxLength, MultiLine, PasswordChar, ListIndex, ListCount, Min/Max	Move, SetFocus, Refresh, ZOrder, Drag, AddItem, RemoveItem, Clear, Cls, Print, Line, Circle, PSet; collections: Add / Remove / Clear	Click, DblClick, Change, KeyPress, KeyDown/KeyUp, MouseDown/MouseUp/MouseMove, GotFocus/LostFocus, Timer, Scroll, DragDrop, DropDown

Control facts

Control	Remember
ImageList	Invisible; ListImages (1-based); supplies pictures to TreeView, ListView, Toolbar, TabStrip, ImageCombo
Toolbar / StatusBar	Buttons collection + ButtonClick(Button) / Panels collection (max 16), Panel.Style (sbrDate, sbrTime …)
ProgressBar / Slider	Min, Max, Value – ProgressBar displays progress; Slider lets the user choose
TabStrip	Tabs collection, Click event; not a container
TreeView / ListView	Nodes + tvwChild, NodeClick / ListItems + ColumnHeaders + View = lvwReport, ItemClick
ImageCombo	ComboItems; ComboBox with pictures
CoolBar	COMCT332.OCX (Common Controls-3); Bands each host one child control; needs IE 3.0+
DriveListBox / DirListBox / FileListBox	Drive; Path; Path + Pattern + FileName – linked through Change events (Dir1.Path = Drive1.Drive)

Likely exam points

- Name and Index are read-only at run time; Caption is for Label/CommandButton/CheckBox/OptionButton/Frame, Text for TextBox/ComboBox.
- CheckBox Value = 0/1/2; OptionButton Value = True/False; ListIndex = –1 when nothing is selected; list indexes start at 0.
- Timer: Interval in milliseconds, Enabled must be True, only the Timer event, no methods, invisible at run time.
- Methods are called, properties are set, events are handled; SetFocus (method) ≠ GotFocus (event).
- KeyPress gives KeyAscii (Enter = 13); KeyDown gives KeyCode; ListBox has no Change event.
- Typing in a TextBox: KeyDown → KeyPress → Change → KeyUp; clicking a button: MouseDown → MouseUp → Click.
- Common Controls 6.0 = MSCOMCTL.OCX (9 controls); Common Controls-3 = COMCT332.OCX = CoolBar.
- Collections of common controls are 1-based; TreeView child node uses tvwChild (4); ListView table uses lvwReport (3).
- Control array: same Name, same type, unique Index (0–32767), shared events with Index argument; Load/Unload at run time; design-time elements cannot be Unloaded.
- Design time: copy–paste (Yes), same Name or Index property. Run time: Load → position → Visible = True → Unload.
- VB 6.0 also allows Set x = Controls.Add("VB.CommandButton", "name") with a WithEvents variable – no control array needed; remove with Controls.Remove.
- Common controls are configured at design time through Property Pages; LoadPicture assigns pictures at run time.